Information Retrieval 2, 227–243 (2000)
© 2000 Kluwer Academic Publishers. Manufactured in The Netherlands.

Comparison and Classification of Documents Based on Layout Similarity

JIANYING HU jianhu@research.bell-labs.com
RAMANUJAN KASHI ramanuja@research.bell-labs.com
GORDON WILFONG gtw@research.bell-labs.com
Lucent Technologies Bell Labs, 700 Mountain Avenue, Murray Hill, NJ 07974-0636, USA

Received April 1, 1999; Revised September 30, 1999; Accepted October 7, 1999

Abstract. This paper describes features and methods for document image comparison and classification at the spatial layout level. The methods are useful for visual similarity based document retrieval as well as fast algorithms for initial document type classification without OCR. A novel feature set called interval encoding is introduced to capture elements of spatial layout. This feature set encodes region layout information in fixed-length vectors by capturing structural characteristics of the image. These fixed-length vectors are then compared to each other through a Manhattan distance computation for fast page layout comparison. The paper describes experiments and results to rank-order a set of document pages in terms of their layout similarity to a test document. We also demonstrate the usefulness of the features derived from interval coding in a hidden Markov model based page layout classification system that is trainable and extendible. The methods described in the paper can be used in various document retrieval tasks including visual similarity based retrieval, categorization and information extraction.

Keywords: hidden Markov models, edit distance, dynamic warping, document classification, document retrieval, clustering, Manhattan distance

1. Introduction

Capturing visual similarity between different document images is a crucial step in many document retrieval tasks, including visual similarity based retrieval, categorization, and information extraction. In this paper we study features and algorithms for document image comparison and classification at the spatial layout level. From the information retrieval point of view, these features and algorithms can be used directly to answer queries based on layout similarities (Cullen et al. 1997, Doermann 1997), or as pre-processing steps for some of the more detailed page matching algorithms (Hull and Cullen 1997, Doermann et al. 1997) for purposes such as duplicate detection. Furthermore, they can serve as the first step in document type identification (business letters, scientific papers, etc), which is useful for both document database management (e.g., business letters might be stored separately from scientific papers), document routing and information extraction (e.g., extracting sender's address and subject from an incoming fax page).

The importance of document layout is demonstrated by figure 1, where on the top are five document pages of different types and on the bottom are the text blocks extracted from each page. Obviously, humans can make a very good judgment as to what type of document

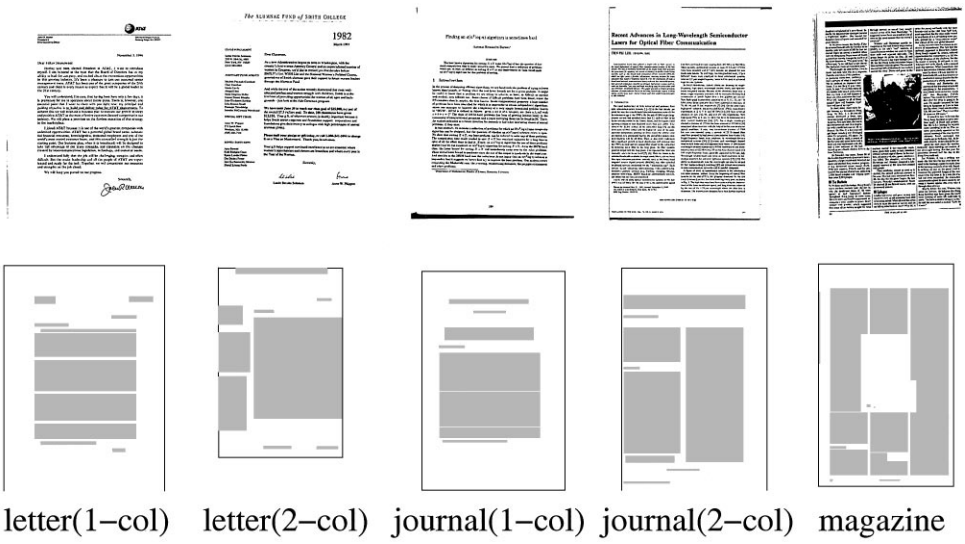

Figure 1. Different types of documents and the corresponding text blocks.

each page represents by looking at the layout image alone, without the help of any textual information. This indicates that layout carries important information about a document. Furthermore, layout characteristics are clearly identifiable at very low resolution. These observations suggest that efficient algorithms can be developed for fast document type classification based on page segmentation only, without going through the time consuming OCR (Optical Character Recognition) process. The results of such classification can then be verified by more elaborate methods using more complicated geometric and syntactic models for each particular class (e.g., Dengel and Dubiel 1996, Taylor et al. 1995, Walischewski 1997).

Measuring spatial layout similarity is in general difficult, as it requires characterization of similar shapes while allowing for variations originating both from design and from imprecision in the low level segmentation process. Zhu and Syeda-Mahmood viewed document layout similarity as a special case of regional layout similarity of general images and proposed a region topology-based shape formalism called constrained affine shape model (Zhu and Syeda-Mahmood 1998). They use an object (region) based approach and attempt to find correspondences, under constrained affine transforms, between regions in different images based on both shape measures of individual regions and spatial layout characteristics. The drawback of this approach is that it depends on low-level segmentation, which may not always be reliable. In the case of document page segmentation, split or merged blocks are fairly common, especially under noisy conditions.

As in many other pattern recognition problems, there is a trade-off between using high-level and low-level representations in layout comparison. A high-level representation such as the one used in Zhu and Syeda-Mahmood (1998) provides a better characterization of the image, but is less resilient to low-level processing errors, and generally requires a more

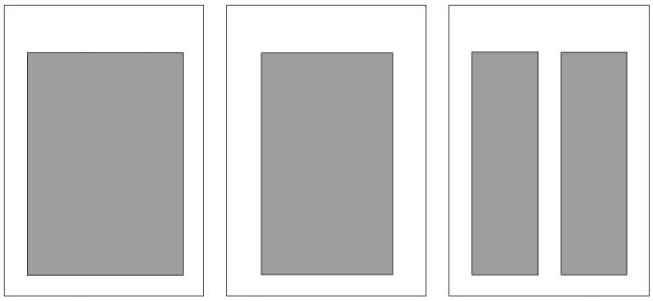

Figure 2. Comparing different layout structures.

complicated distance computation. On the other hand, a low level representation, such as a bit map, is very robust and easy to compute, but does not capture the inherent structures in an image. For example, figure 2 illustrates the different layout structures for three simple pages. Pages 2(a) and 2(b) both have one-column layout, with slightly different dimensions. Page 2(c) has a two-column layout. Clearly pages 2(a) and 2(b) are more similar in a visual sense than pages 2(a) and 2(c). However, a direct comparison of their bit-maps will yield similar distances in both cases.

In this paper, we propose a novel spatial layout representation called *interval encoding*, that can be viewed as an intermediate level of representation. It encodes region layout information in fixed-length vectors, thus capturing structural characteristics of the image while maintaining flexibility. These fixed-length vectors can be compared to each other through a simple distance computation (the Manhattan distance), and thus can be used as features for fast page layout comparison. Furthermore, they can be easily clustered and used in a trainable statistical classifier for document type identification. We present algorithms for page layout comparison using interval encoding. We also demonstrate the effective use of features derived from interval encoding in a hidden Markov model (HMM) based page layout classification system that is trainable and extensible.

2. Page layout comparison

We assume that the input to our layout comparison and classification algorithms are document pages that have been segmented and deskewed. Currently we only look at text blocks and discard other types of content, such as half-tone images and graphics. Therefore, each input is a binary image of text blocks on the background, which we refer to as layout image.

The particular segmentation scheme we have adopted uses white space as a generic layout delimiter and is suitable for machine-printed Manhattan layouts (Baird 1994). The resulting segmentation was found to be reasonable for classes of documents with only text in it. However, in magazine pages with pictures in the middle of the page, the segmentation algorithm resulted in under-segmentation. This was corrected by adding a bottom-up procedure to the segmented blocks which starts by merging evidence at the lowest detail and

then rises, merging words into lines and finally lines into blocks. Examples of the resulting text blocks are shown in the bottom row of figure 1. Notice that this particular page segmentation algorithm generates only rectanglular blocks, however this is not a requirement of our comaparison and classification algorithms, as will become clear later.

As mentioned in the introduction, in choosing a model for layout comparison one is faced with the trade-off between high-level and low-level representations. Our approach is to take the middle ground. First, each layout image is partitioned into an m by n grid and we refer to each cell of the resulting grid as a *bin*. A bin is defined to be a *text bin* if at least half of its area overlaps a single text block and otherwise it is a *white space bin*. In general, we think of a layout image as m rows of n bins where each bin is either labeled as a text bin or a white space bin. We let r_i denote the ith row and index the bins in r_i as $1, \ldots, n$ from left to right. Now our goal is to compare two such images. In comparing two layout image we use a 2-level procedure. The first level consists of a method for computing a distance between one row on one page with another row on the other page. The second level involves finding a correspondence between rows of the two pages that attempts to minimize the sum of distances between corresponding rows.

2.1. Distance computation

The distance between two rows can be computed in numerous ways. We investigate three methods.

It seems clear to us that the most natural method is based on the the following representation of a row of bins. We define a *block* in a row to be a maximal consecutive sequence of text bins. Suppose the ith row, r_i, has k_i different blocks say, $B_1^i, \ldots, B_{k_i}^i$, ordered from left to right. The bins within a particular block B_j^i are consecutive and we let s_j^i and f_j^i denote the index of the first (leftmost) and last (rightmost) bins in B_j^i respectively. We can represent r_i as a sequence of the pairs $R_i = (s_1^i, f_1^i), \ldots, (s_{k_i}^i, f_{k_i}^i)$ and we call such a representation a *block sequence*. Then the distance $d^E(R_i, R_j)$ between two such block sequences R_i and R_j is defined to be the edit distance (see Kruskal and Sankoff 1983) between them where the cost of inserting or deleting a pair (s, f) is just $f - s + 1$ (the width of the block), and the cost of substituting (s, f) with (u, v) is taken to be $|s - u| + |f - v|$. This distance measure will be referred to as the *edit distance*. While the edit distance appears to be the measure that most accurately captures the differences between rows, it can be computationally unattractive if the lengths of the R_i's become too large. The edit distance also has a disadvantage when it comes to clustering. In particular, given a collection of block sequences that form a cluster it is desirable to be able to compute a cluster center, that is, determine a block sequence R that minimizes the sum of the distances from R to each of the block sequences in the cluster. However it is not at all obvious how such a cluster center should be computed. This leaves the unsatisfactory option of choosing as the cluster center one of the block sequences in the given collection that minimizes the sum of the distances to the others.

We introduce a computationally simpler method that overcomes the clustering difficulty of the edit distance. The scheme is based on the Manhattan distance; that is, the l_1 distance

where for two vectors $x = (x_1, \ldots, x_n)$ and $y = (y_1, \ldots, y_n)$ the Manhattan distance between them is given by

$$l_1(x, y) = \sum_{i=1}^{n} |x_i - y_i|.$$

In order to use the Manhattan distance we need to represent each row as a fixed length vector. In fact, each row, r_i, will be represented by a vector of length n $(r_i[1], \ldots, r_i[n])$ where $r_i[j]$ is defined as follows. Define $r_i[j] = 0$ if $j \notin [s_t^i, e_t^i]$ for any t, $1 \leq t \leq k_i$. However if $j \in [s_t^i, e_t^i]$ then either we have that $j = s_t^i + p$ for some p, $0 \leq p < \lceil (e_t^i - s_t^i + 1)/2 \rceil$ or $j = e_t^i - p$ for some p, $0 \leq p < \lfloor (e_t^i - s_t^i + 1)/2 \rfloor$ and in either case $r_i[j] = p + 1$. We call such a vector, an *interval encoding*. Intuitively, $r_i[j]$ represents how far away in r_i the jth bin is from a white space bin. This gives us a fixed length representation that encodes both the position and the width of the individual block intersections with the ith row. The method defined by computing the Manhattan distance between interval encodings will be called the *interval distance* and will be denoted by d^I. Of course, one might ask why not just encode a row by setting $r_i[j]$ to be 1 if it is a text bin and 0 otherwise and perform Manhattan distance computations on such binary representations. However the binary representation fails to capture the notion of similarity between rows that we are concerned with. We wish to have a measure that says that two rows are similar if they have similar block intersection patterns. This notion of similarity is better captured by the Manhattan distance between interval encodings than by the Manhattan distance between binary representations. For example, consider the rows shown in figure 3. Four rows are shown and there are eleven bins per row. The text bins are shown as black and the white space bins are shown as white. For $1 \leq i \leq 4$, the interval encoding $E(row_i)$ and the binary representation $B(row_i)$ of

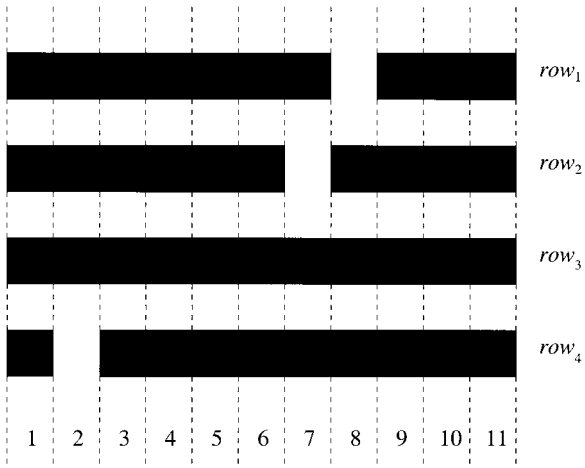

Figure 3. Four examples of rows.

row_i in figure 3 are given by

$$E(row_1) = (1, 2, 3, 4, 3, 2, 1, 0, 1, 2, 1) \quad B(row_1) = (1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1)$$
$$E(row_2) = (1, 2, 3, 3, 2, 1, 0, 1, 2, 2, 1) \quad B(row_2) = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1)$$
$$E(row_3) = (1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1) \quad B(row_3) = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1)$$
$$E(row_4) = (1, 0, 1, 2, 3, 4, 5, 4, 3, 2, 1) \quad B(row_4) = (1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1).$$

In the case shown in the figure, row_1 and row_2 are more similar to each other than either is to row_3 since both row_1 and row_2 intersect two blocks at roughly the same horizontal positions whereas row_3 intersects only one very large block. This is reflected by the fact that the edit distance gives $d^E(row_1, row_2) = 2$ whereas $d^E(row_1, row_3) = 7$ and $d^E(row_2, row_3) = 9$. Using the interval distance we again get that row_1 is more similar to row_2 than either is to row_3 since $d^I(row_1, row_2) = 6$, $d^I(row_1, row_3) = 16$ and $d^I(row_2, row_3) = 18$. However, using d^M, the Manhattan distance between binary representations, gives that $d^M(row_1, row_2) = 2$ whereas $d^M(row_1, row_3) = d^M(row_2, row_3) = 1$. Similarly, although row_1, row_2 and row_4 all intersect exactly two blocks, row_1 and row_2 are more similar to each other than either is to row_4 since they intersect blocks at similar horizontal positions whereas the blocks that row_4 intersects have much different horizontal positions. Again, the edit distance and interval distance agree with this sense of similarity since

$$d^E(row_1, row_4) = d^E(row_2, row_4) = 10 > d^E(row_1, row_2) = 2$$

and

$$d^I(row_1, row_4) = 18 > d^I(row_2, row_4) = 17 > d^I(row_1, row_2) = 6.$$

However, using d^M says that row_1, row_2 and row_3 are each at distance 2 from one another.

Given a collection $E = \{e_1, \ldots, e_t\}$ of interval encodings, we now discuss how they can be partitioned into clusters whose cluster centers can be easily computed. The interval encodings are vectors of length n. A simple and effective means for computing clusters is the k-means clustering technique (Gersho and Gray 1992. This iterative technique requires computing cluster centers for each candidate cluster and for each e_i computing the nearest cluster center to e_i. We define the Manhattan distance to be the distance measure used for these computations. In this case, a cluster center c is defined to be an n-dimensional vector that minimizes the sum of the Manhattan distances from c to each of the elements of the cluster. Notice that if C is a collection of 1-dimensional points, a point p that minimizes the sum of Manhattan distances from p to the points of C is just a point that has the property that the number of elements of C that are greater than or equal to p equals the number of elements of C less than or equal to p. Note that in the case that the cardinality of C is even, there may be a non-trivial closed interval $[a, b]$ of values so that any value in $[a, b]$ will serve equally well in minimizing the sum of the distances and so we break such ties by choosing p to be $\lceil (b-a)/2 \rceil$. The point p is said to be the *median* of the set C. It is easy to see then that for n-dimensional sets C, the cluster center c of C is an n-dimensional vector whose ith component is the median of the ith component of the elements of C. Note that in

general, c will not necessarily be an interval encoding. Thus unlike in the case of the edit distance described above, we can compute a true cluster center since we are not restricted to choosing some element of a cluster to act as the cluster center.

The above discussion shows that given a collection of interval encodings we can use k-means clustering (based on the Manhattan distance measure) to compute clusters and their centers. This leads to a third method for computing the distance between two rows based on their interval encodings. The distance $d^C(r_i, r_j)$ between interval encodings r_i and r_j is defined to be the l_1 distance between c_1 and c_2 where c_1 is the cluster center nearest (in the l_1 sense) to r_i and c_2 is the cluster center nearest to r_j. We will call this distance measure the *cluster distance*.

2.2. Row alignment

In order to capture the visual similarity between document images, we used a dynamic programming algorithm to match rows from one page to the other. The algorithm finds a sub-optimal path by minimizing the total distance cost in aligning the rows on two pages. The feature representation of the ith page is $P_i(l)$. The independent variable l is the index that identifies the row of the page. The mapping which minimizes the distance between two pages denoted by $P_i(l)$ and $P_j(l)$, is the optimal alignment between the respective pages, and this minimal distance is taken as the error D_{ij} defined as

$$D_{ij} = \min_{\lambda}(dist(P_i(l), P_j(\lambda(l))) \tag{1}$$

where *dist* can be one of the three distances measures, d^E, d^I or d^C described in the previous section.

The error D_{ij} is evaluated according to the techniques of dynamic programming. In the application of DP techniques Sakoe and Chiba (1978), a family of mappings $\lambda(l)$ are provided from the rows of the template page to the rows of the test page. At various points, any of these mappings may be many-to-one as well as one-to-many. However, all of these mappings which are often referred to as "warping functions" must be continuous and monotonic.

2.3. Distance comparisons

The warping computation described in Section 2.2 provides a measure of distance between two pages that is dependent on the method used to determine the distance between two rows. We ran experiments using the three inter-row distance measures, the edit distance, the interval distance and the cluster distance as described in Section 2.2. The warping computation with respect to the edit distance is considered to be the method that provides the most natural measure of distance between two pages. Thus the results of this computation are taken to be the "ground truth" in the experiments that are described below. The warping computation with respect to the interval distance is an approximation of this measure and when warping is done with respect to the cluster distance an even more extreme approximation is obtained. We wish to show that neither approximation is likely to be too far from the ground truth. Our goal in the following experiment is to justify this claim.

Let $T_1, \ldots, T_5$ denote the partition of our data into the five document types. That is, each T_i consists of all pages in our data of a particular type. From each T_i we sample a subset S_i of size 10. Let $S = \cup_{i=1}^{5} S_i$. The experiment we perform is as follows. For each $s \in S$ we compute three lists $L^E(s) = (L_1^E(s), \ldots, L_{49}^E(s))$, $L^I(s) = (L_1^I(s), \ldots, L_{49}^I(s))$ and $L^C(s) = (L_1^C(s), \ldots, L_{49}^C(s))$ where each list contains each element of $S \backslash \{s\}$ and the elements of list $L^x(s)$ (where x is one of E, I or C) are ordered so that $d^x(s, L_i^x(s)) \leq d^x(s, L_{i+1}^x(s))$. That is, the lists are ordered so that those pages that are most similar to s appear earlier in the list. In fact, since the focus is on nearest neighbor computations we are only interested in seeing that those pages that are "near" to s (in the d^E sense) remain near to s when using the approximate distance measures. In particular, we will base our measurements on the 5 nearest pages to s as determined by d^E. Thus to compare lists $L^I(s)$ and $L^C(s)$ to $L^E(s)$ we use the following measure. For $x = I$ or $x = C$, let $e(i)$ be the index so that $L_i^x(s) = L_{e(i)}^E(s)$ and similarly let $x(j)$ be such that $L_j^E(s) = L_{x(j)}^x(s)$. That is, $e(i)$ is the position in $L^E(s)$ of the ith element in the list $L^x(s)$ and $x(j)$ is the position in $L^x(s)$ of the jth element of $L^E(s)$. We say that $L_t^x(s)$ and $L_i^E(s)$ are *swapped* if $t < x(i)$ and $i < e(t)$ (that is, the two pages appear in opposite relative order in the two lists). Then define

$$K^x(i,t) = \begin{cases} d_{e(t)}^E(s) - d_i^E(s) & \text{if } L_i^E(s) \text{ and } L_t^x \text{ are swapped} \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

Finally we define the distance from $L^x(s)$ to $L^E(s)$ as

$$D(L^x(s), L^E(s)) = \sum_{i=1}^{5} \sum_{t=1}^{x(i)-1} K^x(i,t). \tag{3}$$

The above definition says that for each of the 5 nearest neighbors $N_1, \ldots, N_5$ of s (in the edit distance sense), find out the position of N_i in the list $L^x(s)$ and for every element appearing before N_i in the list $L^x(s)$ we "charge" the amount given by K^x for every one which is swapped with N_i.

To illustrate the intuition behind the definition of D the distance measure between ranking lists consider figure 4 where we show the ranking list $L^E(s)$ due to the edit distance (the ground truth) and the ranking list $L^C(s)$ due to the cluster distance. There are five pages in the experiment and in this case we are interested in the two nearest neighbors to sample page s. The pages are numbered according to their ranking in $L^E(s)$. Computing $D(L^C(s), L^E(s))$ we note that page 2 and page 4 have been ranked higher in $L^C(s)$ than page 1. Also, page 4 has been ranked higher than page 2 in $L^C(s)$. The distance $D(L^C(s), L^E(s))$ will consist of terms due to the pages ranked higher than page 1 and those due to the pages ranked higher than page 2 in list $L^C(s)$. For example, the term due to page 4 being ranked higher than page 2 is set to be $d_4^E - d_2^E$ because this gives a measure of how "strongly" the ground truth differentiated between the two pages. That is, if the two pages were thought to be nearly equal in distance from the sample page s by the ground truth (i.e. $d_4^E - d_2^E$ is small) then having them swapped in $L^C(s)$ should not be too heavily penalized and similarly if the ground truth indicated that page 4 was much further from s than page 2 was (i.e. $d_4^E - d_2^E$

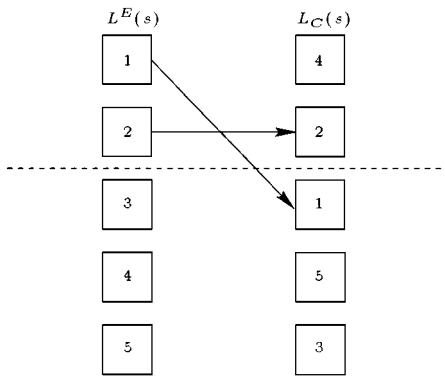

Figure 4. Comparing cluster ranking with ground truth.

is large) then the cluster ranking should be more heavily penalized. Therefore the ranking score for the cluster distance is $D(L^C(s), L^E(s)) = (d_2^E - d_1^E) + (d_4^E - d_1^E) + (d_4^E - d_2^E)$.

2.4. Experimental results

The objective of this preliminary experiment was to examine the effectiveness of the proposed distance measures by rank-ordering a set of documents in terms of their similarity to a test document. The document database consisted of samples of one column letters, two column letters, one column journal pages, two column journal pages and magazine articles, as shown in figure 1. Examples of two column letters are usually seen in business type letters where one narrow column often consists of lists of items, for example, a list of names of partners in a law firm or a list of the company's locations, and the other column represents the body of the letter. Each of these five classes had ten samples and thus a total of fifty documents were used in this experiment. All of the samples were monochrome page images at 300 dpi.

The experiment consisted of choosing a document at random as a test sample from this set of fifty and rank-ordering the remaining documents in terms of its similarity with respect to the test document. The similarity was computed based on the three measures described in the previous section. The first was similarity based on edit-distances. The second one was based on the interval distance using the interval encoding vectors extracted from the documents and the test sample. The third was based on the cluster distance using the clustering results of the extracted interval encoding vectors. Since during our tests edit-distance consistently produced rankings that matched human judgements very well, we decided to regard these rankings as the ground truth, against which the rankings produced by the interval distance and cluster distance are compared. In order to get a more concrete idea as to how well the proposed measures performed, we introduce a more naive row distance measure to rank-order the documents and show how the interval distance and cluster distance based rankings outperform this new measure. The naive row distance measure is obtained by direct bit-wise xor of the corresponding rows.

Table 1. Ranking scores.

	S_{interval}	S_{cluster}	S_{bitmap}
Magazine	556	944	16529
1-col-journal	40	288	22538
2-col-journal	2392	3075	29757
1-col-letter	118	258	1755
2-col-letter	2831	4119	21311

Figure 5 shows the first five ranks of documents matched with the test sample shown at the top. The test sample shown in the figure is of a two-column journal type of document. Under the edit distance column are the five documents obtained from the database which are ranked according to their closeness to the test same in the edit-distance sense. Under the interval distance column, are the five documents obtained in terms of the interval distance sense. The five documents retrieved also appears in the edit-distance column, albeit, in a different order. Note that the first image aligns with the first in the edit-distance column. The second and third (similarly the fourth and fifth) in the interval-distance column have been interchanged with respect to the edit-distance column. Using the clustering technique described in the previous sections the top five documents retrieved appear similar to the test sample and are also present in the edit-distance column. Under the cluster-distance column, the top image is the same as that of the top in the edit-distance column. The second image in this column appears as the fourth similar image in the edit-distance column. Similarly, the third and the fourth in the cluster-distance column appear as the second and third respectively in the edit-distance column. Finally, the crude representation of the bit-map distance gets four one-column journal articles in the top five choice for being closest with the two-column journal article. As seen in figure 5 the two distance measures, obtained from the interval encoding and the clustered version of the encoding, capture the notion of "similarity" and obtain ranks similar to that of the edit-distance scheme.

The experiment was repeated by taking an example from each of the five classes as a test sample and ranking the remaining forty-nine documents. The ranks obtained from different schemes (edit-distance, interval-distance, cluster-distance, bitmap-distance) were compared using the scheme explained in Section 2.3. Table 1 shows the ranking scores (S_{interval}, S_{cluster}, S_{bitmap}) obtained while comparing the first ten rankings between one of the ranking schemes (interval, cluster, bitmap) and that of the edit-distance (ground-truth). Note that the scores, S_{interval} and S_{cluster} are close which points to the observation made earlier that the feature does not degrade severely with clustering. The bit-map comparison score, S_{bitmap}, is significantly higher as seen in Table 1. This was expected since the bit-map representation of the blocks does not capture the inherent structure of a row of text as seen earlier in figure 3.

3. Layout classification

We now describe the application of interval encoding features for layout classification. Layout classification can be used to achieve more efficient organization and retrieval of

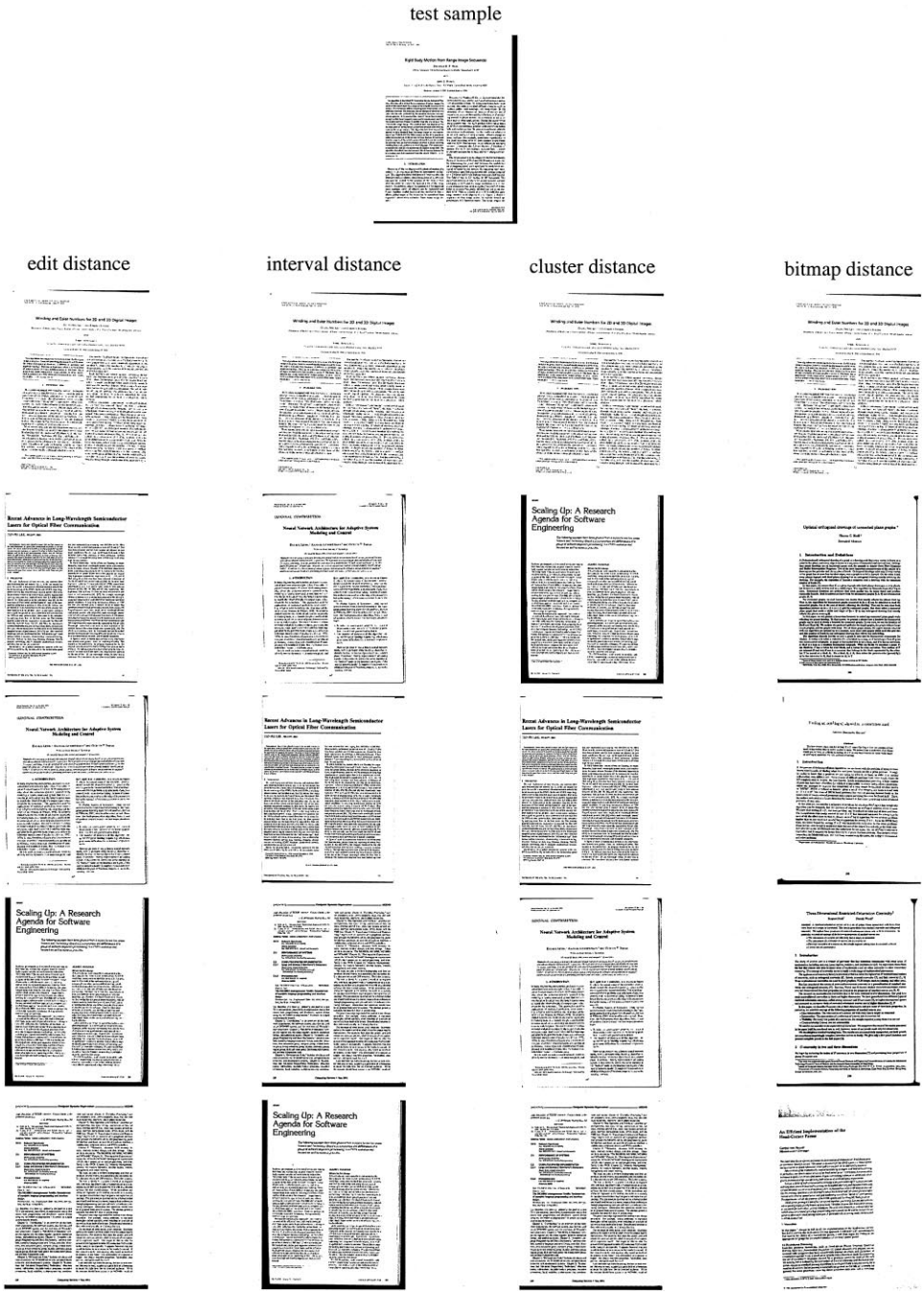

Figure 5. Page layout similarity ranking using different distance measures.

documents based on layout similarity. It can also serve as a fast initial stage for document type classification, which is an important step for document routing and understanding. For example, if an incoming fax page is classified as a potential business letter, more detailed logical analysis combining geometric features and text cues derived from OCR can then be applied to confirm the hypothesis and assign logical labels to various fields such as sender address, subject, date, etc., which can then be used for routing or content based retrieval and feedback to the user. The advantage of using layout information only in the initial classification is that it is potentially very fast since it does not rely on any OCR results.

We chose to use Hidden Markov Models (HMM) to model different classes of layout designs for several reasons. First, from our observation, a particular layout type is best characterized as a sequence of vertical regions, each region having more or less consistent horizontal layout features (i.e., number of columns and widths and position of each column) and a variable height. Thus a top-to-bottom sequential HMM, where the observations are the interval encoding features described in the previous section and the states correspond to the vertical regions, seems to be the ideal choice of model. Second, HMMs have been well studied and have proven to be very effective in modeling other stochastic sequences of signals such as speech or on-line handwriting (Rabiner and Juang 1993, Hu et al. 1996). Because of the probabilistic nature, they are robust to noise and thus are well suited to handle the inevitable inconsistencies in low-level page segmentation (e.g., vertical splitting or merging of blocks). Furthermore, there exist well established efficient algorithms for both model training and classification (Rabiner and Juang 1993).

3.1. Model descriptions

The state transition diagram of a top-to-bottom sequential HMM in the classic form is shown in figure 6 (it is drawn left to right here for convenience). A discrete model with N states and M distinct observation symbols $v_1, v_2, \ldots, v_M$ is described by the state transition probability matrix $\mathbf{A} = [a_{ij}]_{N,N}$, where $a_{ij} = 0$ for $j > i + 1$, state conditional observation probabilities: $b_j(k) = Pr(O_t = v_k \mid s_t = j)$, $k = 1 \ldots M$, and initial state distribution: $Pr(s_1 = 1) = 1$, $Pr(s_1 = i) = 0$ for $i > 1$. We use cluster labels as observations and thus the number of distinct observation symbols M equals the number of clusters, which roughly represents the number of distinct horizontal layout patterns seen in the training set. The number of states should correspond to the maximum number of major vertical regions in a class, and should ideally vary from class to class. However for our initial experiments we used a fixed number of 20 states for all classes. Currently M is chosen empirically to be 30.

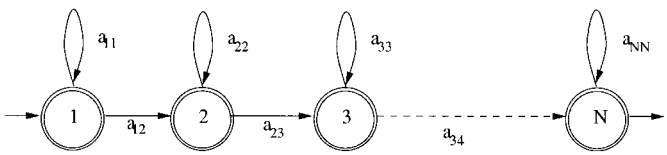

Figure 6. Top-to-bottom HMM state-transition diagram.

One constraint of the classic HMM described above is that state duration (state-holding time) distribution always assumes an exponential from. It is easy to verify that the probability of having d consecutive observations in state i, is implicitly defined as:

$$p_i(d) = (a_{ii})^{d-1}(1 - a_{ii}). \tag{4}$$

In our modeling scheme where states correspond to vertical layout regions and each region may assume a range of possible lengths with similar probabilities, this distribution is inappropriate. In order to allow more flexibility we chose to use a mechanism called explicit duration modeling, resulting in a variable-duration HMM which was first introduced in speech recognition (Ferguson 1980, Levinson 1986) and later proved effective for signature verification as well (Kashi et al. 1997). This model can also be called a Hidden Semi-Markov Model (HSMM), because the underlying process is a semi-Markov process (Turin 1990). In this approach, state-duration distribution $p_i(d)$ can be a general discrete probability distribution. It is easy to prove that any variable duration HMM model is equivalent to the so-called canonical HSMM in which $a_{ii} = 0$ (Turin 1990). Thus, for the variable-duration HMM there is no need of estimating state transitional probabilities.

The downside of allowing $p_i(d)$ to be any general distribution is that it has many more parameters and thus requires many more training samples for a reliable estimation. To alleviate this problem we impose a simplifying constraint on $p_i(d)$ such that it can only assume a rectangular shape. Under this constraint, only the duration boundaries τ_i and T_i ($\tau_i < T_i$) are estimated for each state during training. Then the duration probabilities are assigned as: $p_i(d : d > T_i \text{ or } d < \tau_i) = \delta$; $p_i(d : \tau_i \leq d \leq T_i) = (1 - \delta)/(T_i - \tau_i + 1)$, where δ is a small constant. One might point out that with this simplification, we have now replaced one assumption on the shape of p_i (exponential) with another (rectangular). However our experiments show that the latter indeed produces much better results, verifying that it is a more appropriate assumption for page layout models.

We use the Viterbi algorithm for both training and classification as opposed to the more rigorous Baum-Welch and forward-backward procedures because the former can easily accommodate explicit duration modeling without much extra computation (Rabiner and Juang 1993). The Viterbi algorithm searches for the most likely state sequence corresponding to the given observation sequence and gives the accumulated likelihood score along this best path. Using explicit state duration modeling, the increment of the log-likelihood score for the transitions to the $i + 1$-st state are given by

$$\Delta L_{i,i+1} = \log b_{i+1}(k) + p_i(d) \tag{5}$$

$$\Delta L_{i+1,i+1} = \log b_{i+1}(k) \tag{6}$$

It should be pointed out that with explicit duration modeling the Viterbi algorithm is no longer guaranteed to find the optimal state sequence, because now the accumulated score of a sequence leading to state i not only depends on the previous state, but also on *how* the previous state was reached (history reflected in the duration d). This dependence violates the basic condition for the Viterbi algorithm to yield optimum solution. However, our

experiments showed that the gain obtained by incorporating explicit duration modeling by far outweighs the loss in the optimality of the algorithm.

The HMM of each layout class is trained by applying a segmental k-means iterative procedure Rabiner and Juang (1993) across all the training samples in the class. The procedure is composed of iterations of the following 2 steps:

1. Segmentation of each training sample by the Viterbi algorithm, using the current model parameters.
2. Parameter re-estimation using their means along the path.

The initial model parameters are obtained through equal-length segmentation of all the training samples. The re-estimation stops when the difference between the likelihood scores of the current iteration and those of the previous one is smaller then a threshold (usually after 3–5 iterations).

During classification, first preprocessing and feature extraction are carried out and the observation sequence is computed for a given page. Then the Viterbi algorithm is applied to find the likelihood of the the observation sequence being generated by each one of the K class models. The page is then labeled as a member of the class with highest likelihood score. Alternatively, more than one class with highest likelihood score can be retained as potential class labels to be examined more carefully later on.

3.2. Experiments

Preliminary experiments were conducted to classify a test document into one of five classes. The five classes of documents used in the study were one-column journal articles, two-column journal articles, one-column letters, two-column letters and magazine articles. Thirty samples in each class were used to build a Hidden Markov model template for that class. A test of ninety-one monochrome document samples at 300 dpi were used in the classification experiment. Each test sample was then compared with the five models to generate likelihood scores. The results of the classification for the test samples is shown in Table 2. Each row of the table corresponds to one of the five classes of documents.

As seen in Table 2, sixteen of the 1-column-journals obtained the highest likelihood scores when compared against the template of 1-column journal class and hence were correctly classified. Six of the test samples which were 1-column journal articles were incorrectly

Table 2. Classification results on the five classes of documents.

	1-col-journal	2-col-journal	1-col-letter	2-col-letter	Magazine
1-col-journal	16	0	6	0	0
2-col-journal	0	18	0	0	0
1-col-letter	3	0	7	0	0
2-col-letter	0	2	0	7	2
Magazine	0	4	0	2	24

Table 3. Percentage accuracy in classification of test documents.

	Accuracy_1	Accuracy_2
1-col-journal	73	100
2-col-journal	100	100
1-col-letter	70	100
2-col-letter	64	90
Magazine	86	100

Accuracy_1 relates to accuracy using only the top scores. Accuracy_2 relates to the correct document occurring in either the first or second set of scores (top-two).

classified as 1-column letters. A closer look at the two document classes, 1-column journal articles and 1-column letter articles, reveals that their spatial layouts are quite similar. Differences between these two classes are more apparent at the top of the page. In letters, the top part of the page has address blocks and in journals, it is usually a title with the author information. Thus accurate discrimination between these two classes requires more detailed geometrical/syntactical analysis incorporating OCR results. Similar observations occur in the third row showing the results of classification with one-column letters. In this case seven of the test documents were classified correctly and three of the test samples had the highest likelihood score when compared with the template of a one-column journal article.

The accuracy for each of the five classes of documents is shown in Table 3. As explained earlier, in the classification experiment, each test sample was compared with five models (classes). $Accuracy_1$ refers to the percentage of documents whose correct class appeared as the top-choice in the ranking experiment (top-one accuracy). $Accuracy_2$ refers to the percentage of documents whose correct class appeared in either the top or the second choice (top-two accuracy). These accuracy measures correspond to the recall and precision measures often used in the information retrieval community.

As seen from Table 3, good accuracy scores are achieved in classification based solely on spatial characteristics and without OCR. In particular, the top-two accuracy scores are very high for all but one class. This demonstrates the effectiveness of the method for the screening purpose: with almost perfect recall, one only needs to look at two classes now as opposed to the original five. The accuracies for the two-column letter class are relatively low. We believe the reason is that the samples we have collected for that class include many commercial "junk mails" with highly varied layouts and fancy graphics, causing more errors in segmentation. As a result, the model for that class is relatively weak and thus the two-column letter test samples are more likely to be mis-classified.

4. Conclusions and future work

This paper describes algorithms for visual document similarity comparison based on information related to spatial layout. Based on a novel feature called interval encoding, we

designed and tested algorithms to rank-order document pages based on spatial similarity. Experiments were also carried out to classify document pages in terms of document types using these new features in an HMM framework. The results in the ranking experiment justify the validity of the features introduced. As seen from the results of the classification experiments, reasonably high accuracy scores are obtained based solely on spatial layout and without OCR. These experiments demonstrate that the features and methods introduced in this paper provide an efficient and effective mechanism for document visual similarity comparison as well as initial classification of documents. Results from this initial analysis can be used by later stages of document processing, including use of class specific geometrical and syntactic models incorporating OCR results, to achieve better classification as well as content understanding and extraction.

Much work remains to be done. We have found out in our ranking experiments that edit distance produces similarity rankings that match human perception very well. Carefully designed subjective experiments need to be carried out to verify this observation. We would also like to carry out more classification experiments involving more types of documents as well as more samples per type to verify the results obtained from our preliminary experiments. There are some interesting questions to be answered to expand the current algorithms. For example, how does the algorithm work for documents containing vertical lines as opposed to horizontal lines? Should one use vertical interval encoding and then carry out page matching and modeling along the horizontal direction? Another, more difficult, question is how to incorporate multiple block types (e.g., text of different font sizes, half-tone images, graphics, etc.) in the interval encoding scheme.

Acknowledgments

We would like to thank Henry Baird and John Hobby for the numerous helpful discussions.

References

Baird HS (1994) Background structure in document images. Int Journal of Pattern Recognition and Artificial intelligence, 8(5):1013–1030.

Cullen JF, Hull JJ and Hart PE (1997) Document image database retrieval and browsing using texture analysis. In: Proc. ICDAR'97, Ulm, Germany, pp. 718–721.

Dengel A and Dubiel F (1996) Computer understanding of document structure. Int Journal of Imaging Systems and Technology, 7:271–278.

Doermann D (1997) The retrieval of document images: a brief survey. In: Proc. ICDAR'97, Ulm, Germany, pp. 945–949.

Doermann D, Li H and Kia D (1997) The detection of duplicates in document image databases. In: ICDAR'97, Ulm, Germany, pp. 314–318.

Ferguson JD (1980) Variable duration models for speech. In: Proc. Symp. on the Application of HMM to Text and Speech, Priceton, NJ, pp. 143–179.

Gersho A and Gray RM (1992) Vector Quantization and Signal Compression. Kluwer Academic Publishers.

Hu J, Brown MK and Turin W (1996) HMM based on-line handwriting recognition. IEEE PAMI, 18(10):1039–1045.

Hull JJ and Cullen JF (1997) Document image similarity and equivalence detection. In: ICDAR'97, Ulm, Germany, pp. 308–312.

Kashi R, Hu J, Nelson W and Turin W (1997). On-line handwriting signature verification using hidden Markov model features. In: Proc. ICDAR'97, Ulm, Germany.

Kruskal JB and Sankoff D (1993), Eds. Time Warps, String Edits, and Macromolecules: The Theory and Practice of Sequence Comparison. Addison-Wesley, Reading, MA.

Levinson SE (1986) Continuously variable duration hidden Markov models for automatic speech recognition. Computer Speech & Language, 1(1):29–45.

Rabiner LR and Juang BH (1993) Fundamentals of Speech Recognition. Prentice Hall, Englewood Cliffs, NJ.

Sakoe H and Chiba S (1978) Dynamic programming algorithm optimization for spoken word recognition. IEEE Trans. Acoust., Speech, Signal Processing, ASSP-26:43–49.

Taylor SL, Lipshutz M and Nilson RW (1995) Classification and functional decomposition of business documents. In: Proc. ICDAR'95, Montreal, Canada, pp. 563–566.

Turin W (1990) Performance Analysis of Digital Transmission Systems. Computer Science Press, New York.

Walischewski H (1997) Automatic knowledge acquisition for spatial document interpretation. In: ICDAR'97, Ulm, Germany, pp. 243–247.

Zhu W and Syeda-Mahmood T (1998) Image organization and retrieval using a flexible shape model. In: IEEE Int. Workshop on Content Based Access of Image and Video Databases, Bombay, India, pp. 31–39.